Análisis del problema:
El problema consta de cómo ordenar un conjunto de amigos y si pueden quedar en parejas o en solitario y el número y las posibilidades posibles que hayan.

	Requerimientos
01	El sistema debe permitir ingresar un número entero positivo.
02	No debe aceptar números negativos ni decimales.
03	Debe mostrar un mensaje de error si la entrada es inválida.
04	El resultado se debe mostrar en pantalla.
05	Debe funcionar sin importar el tamaño de n.
06	Debe evitar cálculos repetidos.

ID	Historia de Usuario	Actor	Descripción
HU01	Ingresar un valor	Sistema	El sistema asigna un valor de n el cual va a ser un entero positivo.
HU02	Validar datos ingresados	Sistema	Verifica que la entrada cumpla con los requisitos y debe calcular el resultado.
HU03	Mostrar resultado	Sistema	El sistema debe haber evaluado con programación dinámica y mostrar el resultado en pantalla

Diagrama de secuencia

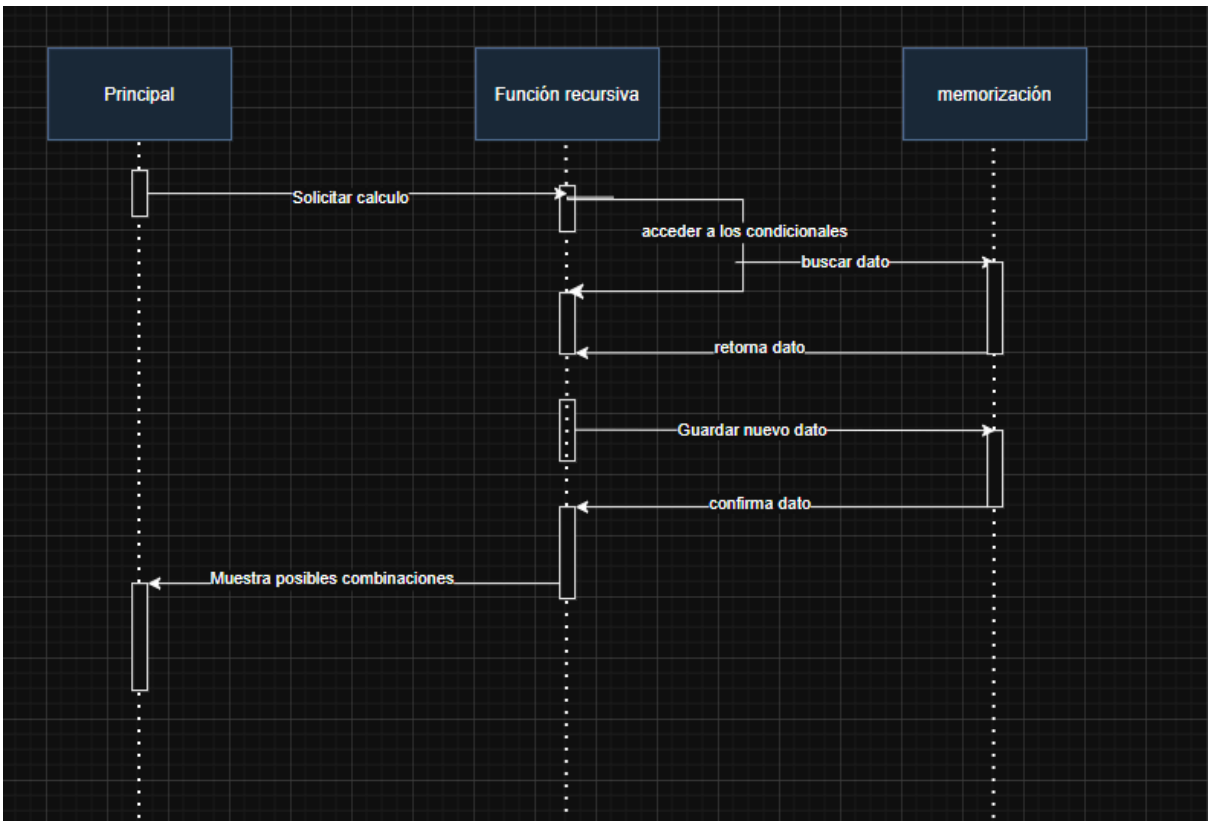
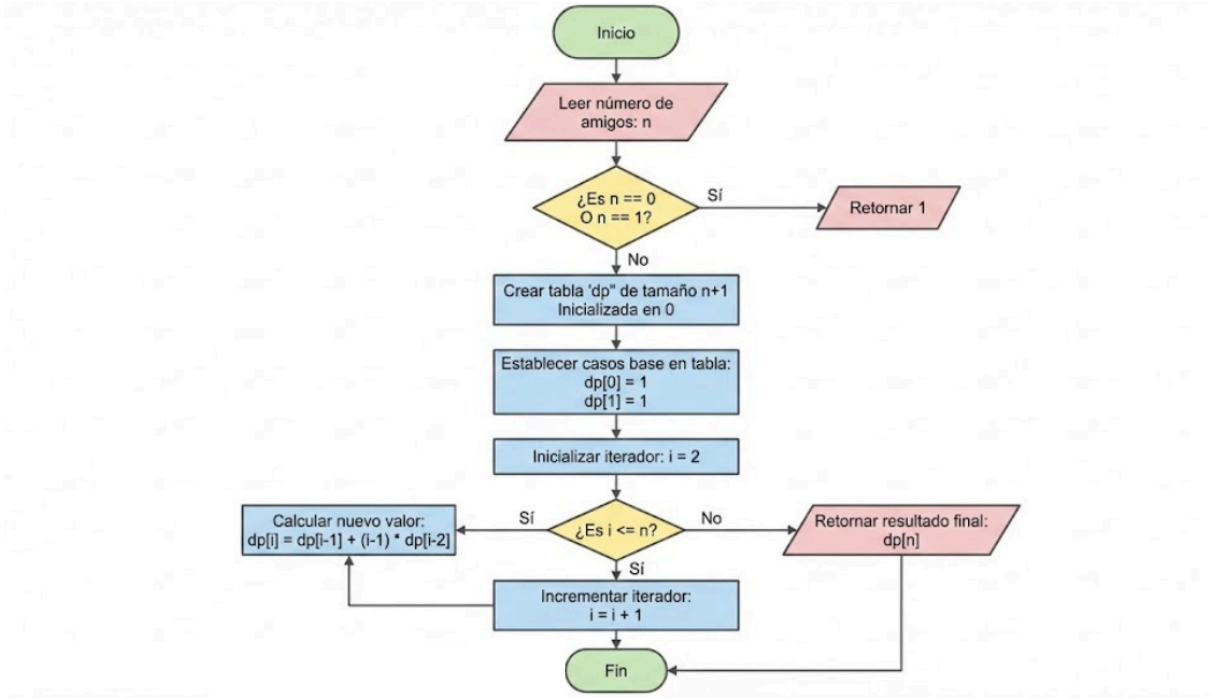
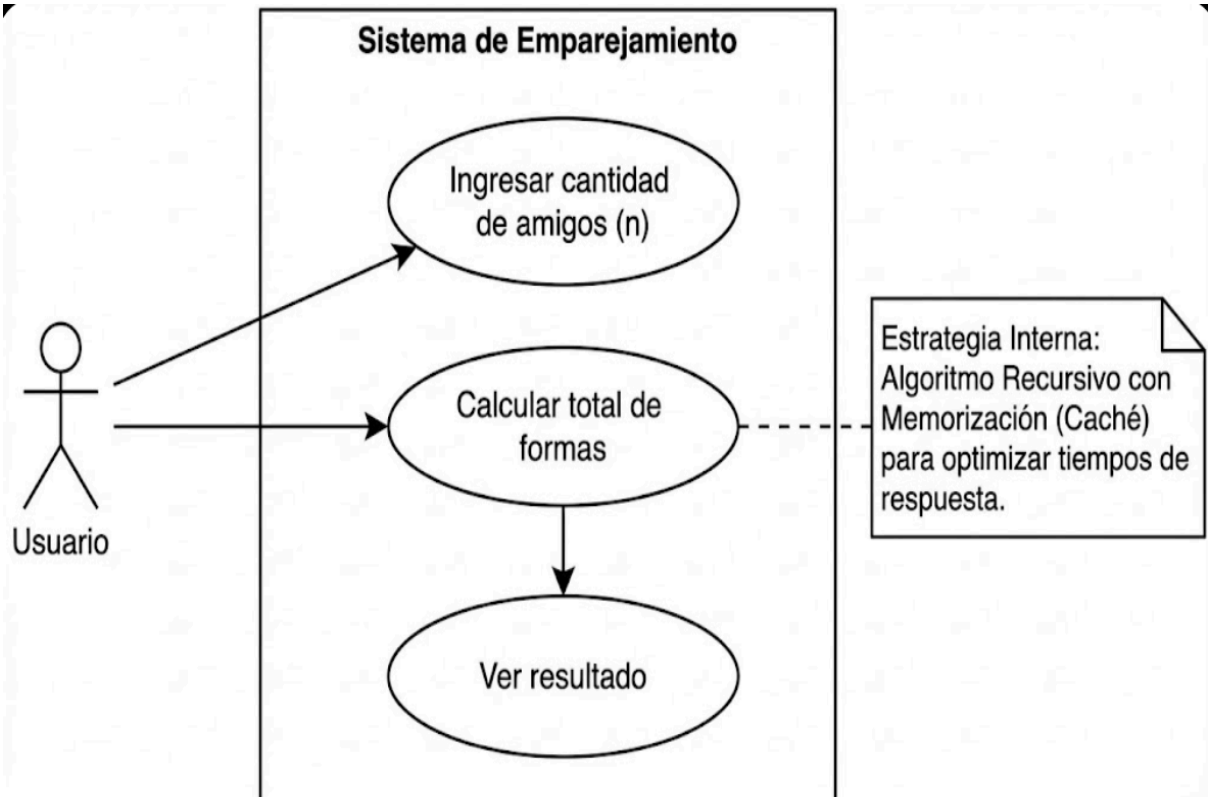


Diagrama de flujo



Caso de uso



Código

codigoexpo3.py > contar_formas_amigos

```python
def contar_formas_amigos(n):
    if n == 0 or n == 1:
        return 1
    dp = [0] * (n + 1)

    dp[0] = 1
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + (i - 1) * dp[i - 2]

    return dp[n]
numero_amigos = 3
resultado = contar_formas_amigos(numero_amigos)
print(f"Para {numero_amigos} amigos, existen {resultado} formas posibles de agruparlos.")
print(f"Para 4 amigos, existen {contar_formas_amigos(4)} formas.")

def contar_formas_amigos_optimizado(n):
    if n == 0 or n == 1:
        return 1

    a = 1
    b = 1

    for i in range(2, n + 1):
        c = b + (i - 1) * a
        a = b
        b = c
    return b
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```
Para 3 amigos, existen 4 formas posibles de agruparlos.
Para 4 amigos, existen 10 formas.
PS C:\Users\danib\OneDrive\Documentos\DDYA>
```

Lín. 5, col. 1 Espacios: 4 UTF-8 CRLF { } Python 3.12.10 (Microsoft Store) Go Live Prettier

Analisis de solucion

El ejercicio implementa la programación dinámica usando un arreglo tipo listo para guardar los resultados previos. El programa verifica los casos base y luego entra a un bucle For que itera desde 2 hasta n.
Complejidad de Tiempo: $O(n)$ lo que representa una complejidad lineal